```
-- ========================================
-- SQL JOINS - COMPLETE EXAMPLE
-- ========================================

-- STEP 1: Create Student Table

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    course_id INT
);

-- STEP 2: Insert Data into Student Table

INSERT INTO Student (student_id, student_name, course_id)
VALUES
(1, 'Anu', 101),
(2, 'Rahul', 102),
(3, 'Sneha', 103),
(4, 'Akhil', 101),
(5, 'Meera', 105);

-- View Student Table

SELECT * FROM Student;

-- ========================================

-- STEP 3: Create Course Table

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50)
);

-- STEP 4: Insert Data into Course Table

INSERT INTO Course (course_id, course_name)
VALUES
(101, 'Python'),
(102, 'SQL'),
(103, 'Power BI'),
(104, 'Excel');
```

```
-- View Course Table

SELECT * FROM Course;

-- ========================================
-- INNER JOIN
-- Returns only matching records
-- ========================================

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
INNER JOIN Course c
ON s.course_id = c.course_id;

-- ========================================
-- LEFT JOIN
-- Returns all records from Student table
-- ========================================

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
LEFT JOIN Course c
ON s.course_id = c.course_id;

-- ========================================
-- RIGHT JOIN
-- Returns all records from Course table
-- ========================================

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
RIGHT JOIN Course c
ON s.course_id = c.course_id;

-- ========================================
```

```
-- FULL OUTER JOIN
-- (Works in PostgreSQL / SQL Server)
-- ========================================

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
FULL OUTER JOIN Course c
ON s.course_id = c.course_id;

-- ========================================
-- FULL OUTER JOIN IN MYSQL
-- ========================================

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
LEFT JOIN Course c
ON s.course_id = c.course_id

UNION

SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM Student s
RIGHT JOIN Course c
ON s.course_id = c.course_id;

-- ========================================
-- CROSS JOIN
-- Returns every possible combination
-- 5 Students × 4 Courses = 20 Rows
-- ========================================

SELECT
    s.student_name,
    c.course_name
FROM Student s
```

```
CROSS JOIN Course c;

-- ========================================
-- PRACTICE QUERIES
-- ========================================

-- 1. Students enrolled in Python

SELECT
    s.student_name,
    c.course_name
FROM Student s
INNER JOIN Course c
ON s.course_id = c.course_id
WHERE c.course_name = 'Python';

-- ========================================

-- 2. Students without a matching course

SELECT
    s.student_name
FROM Student s
LEFT JOIN Course c
ON s.course_id = c.course_id
WHERE c.course_id IS NULL;

-- ========================================

-- 3. Courses without any students

SELECT
    c.course_name
FROM Course c
LEFT JOIN Student s
ON c.course_id = s.course_id
WHERE s.student_id IS NULL;

-- ========================================

-- 4. Count students in each course

SELECT
    c.course_name,
```

```
    COUNT(s.student_id) AS Total_Students
FROM Course c
LEFT JOIN Student s
ON c.course_id = s.course_id
GROUP BY c.course_name;

-- ========================================

-- 5. Display all students and their courses
-- ordered by student name

SELECT
    s.student_name,
    c.course_name
FROM Student s
LEFT JOIN Course c
ON s.course_id = c.course_id
ORDER BY s.student_name;
```

Activity 1

Display the customer name and the product they ordered.

Solution

```
None
SELECT
   c.customer_name,
   o.product
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Activity 2

Display all customers along with their orders.

Solution

```
None
SELECT
   c.customer_name,
   o.product
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id;
```

Activity 3

Display all orders, even if the customer details are not available.

Solution

```
None
SELECT
   c.customer_name,
```

```
   o.product
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id;
```

Activity 4

Find customers who have **not placed any orders**.

Solution

```
None
SELECT
   c.customer_name
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Activity 5

Find orders that **do not have a matching customer**.

Solution

```
None
SELECT
   o.order_id,
   o.product
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Activity 6

Display the customer name, product, and order amount.

Solution

```
None
SELECT
   c.customer_name,
   o.product,
   o.amount
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Activity 7

Display customers who purchased a **Laptop**.

Solution

```
None
SELECT
   c.customer_name,
   o.product
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.product = 'Laptop';
```

Activity 8

Calculate the total purchase amount for each customer.

Solution

```
None
SELECT
   c.customer_name,
   SUM(o.amount) AS Total_Amount
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Activity 9

Display customers whose total purchase amount is greater than **₹30,000**.

Solution

```
None
SELECT
   c.customer_name,
   SUM(o.amount) AS Total_Amount
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING SUM(o.amount) > 30000;
```

Activity 10

Display every possible combination of customers and products.

Solution

```
None
SELECT
   c.customer_name,
   o.product
FROM Customers c
```

```
CROSS JOIN Orders o;
```

Bonus Activities

1. Count the number of orders placed by each customer.

```
None
SELECT
   c.customer_name,
   COUNT(o.order_id) AS Total_Orders
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

---

2. Display customers from Chennai who have placed orders.

```
None
SELECT
   c.customer_name,
   o.product
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id
WHERE c.city = 'Chennai';
```

---

3. Find the highest order amount.

```
None
SELECT
   c.customer_name,
   MAX(o.amount) AS Highest_Order
FROM Customers c
```

```
INNER JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```